

설비 고장을 예측하는 AI 데이터 분석

Git 기초 + GitHub 연동

Git과 GitHub 기초

Git

도구 소개 Part 1

버전 관리와 Git	4
포트폴리오와 Git	10

개발 환경 설정 Part 2

GitHub 가입	13
Git 설치 및 초기 설정	15

Git 사용하기 Part 3

Git 작업 흐름 (add · commit · log)	22
.gitignore와 실습	29

GitHub Part 4

로컬 vs 원격 저장소	33
push · pull · clone	38

GitHub Desktop Part 5

저장소 추가와 커밋	57
Push/Pull과 CLI 비교	62

PART 1

도구 소개

posco

버전 관리란?



파일이나 코드의 변경 이력을 기록하고 관리하는 시스템

언제, 누가, 무엇을, 왜 바꿨는지 추적할 수 있음

이전 버전으로 되돌리거나 여러 버전을 비교·병합 가능

버전 관리 문제 - 파일명 지옥 체험

처음엔 project.html 하나 → 수정할 때마다 backup, final, 진짜최종 붙어 파일 폭발

어느 파일이 최신인지 알 수 없고 무엇이 왜 바뀌었는지 알 방법도 없음

여러 명이 같은 파일을 수정하면 누구 버전이 맞는지 합칠 방법이 없음



버전 관리 문제 - 세 가지 치명적 한계

최신 버전 불명확

파일명과 날짜만으로 어느 것이 진짜 최신인지 판단 불가

- 📄 최종결과.txt
- 📄 최종결과_updated.txt
- 📄 최종결과_진짜진짜최종.txt
- 📄 최종결과_진짜최종.txt

변경 이유 추적 불가

무엇이 왜 바뀌었는지 직접 열어 비교해야만 파악 가능

협업 시 충돌

A와 B가 각자 수정한 파일을 합칠 방법이 없음

Git이 해결하는 것 - 스냅샷과 히스토리

Git은 파일 변경 이력을 **스냅샷**으로 저장하는 버전 관리 시스템

스마트폰 사진 앨범처럼, 코드를 저장할 때마다 그 시점의 전체 상태를 기록

1. 언제 변경됐는지
2. 누가 변경했는지
3. 어떤 내용이 바뀌었는지

Git이 해결하는 것 - 핵심 가치

변경 이력 추적

언제든 되돌리기

Git vs 폴더 복사 백업 - 비교

비교 항목	폴더 복사 방식	Git
변경 내용 파악	직접 파일 열어 비교	자동 추적, 줄 단위 기록
변경 이유 기록	알 수 없음	커밋 메시지로 기록
저장 공간	전체 파일 복사	변경분만 저장
과거로 되돌리기	폴더 찾아 수동 복원	명령어 하나로 즉시

포트폴리오?

3,177 contributions in the last year

Contribution settings ▾



git과 연동한 GitHub **활동 기록**과 **개인 포트폴리오**는 개발자의 실력을 보여주는 가장 좋은 자료입니다.

이번 시간에 사용할 도구



Git · GitHub · Visual Studio Code 세 가지를 함께 다룹니다.

PART 2

개발 환경 설정

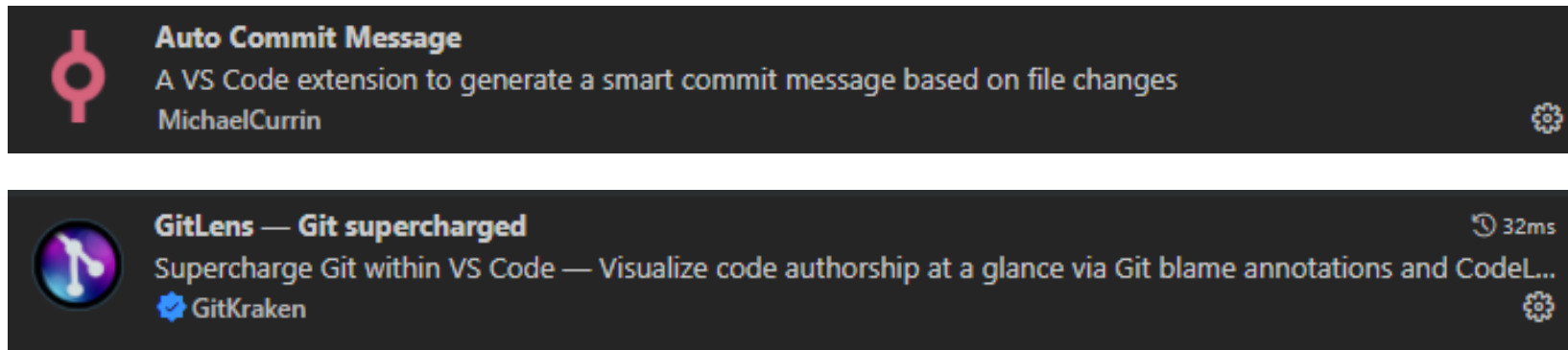
posco

GitHub 회원가입 / 로그인



<https://github.com/>

git을 위한 VS Code 확장 프로그램



Auto Commit Message: 변경 파일을 분석해 커밋 메시지 자동 생성

GitLens: VS Code 안에서 Git 활동을 시각적으로 확인

Git 설치 (Windows)



<https://git-scm.com/>

Download for Windows

[Click here to download](#) the latest (2.36.1) 64-bit version of Git for Windows. This is the most recent maintained build. It was released 24 days ago, on 2022-05-09.

Other Git for Windows downloads

Standalone Installer

[32-bit Git for Windows Setup.](#)

[64-bit Git for Windows Setup.](#)

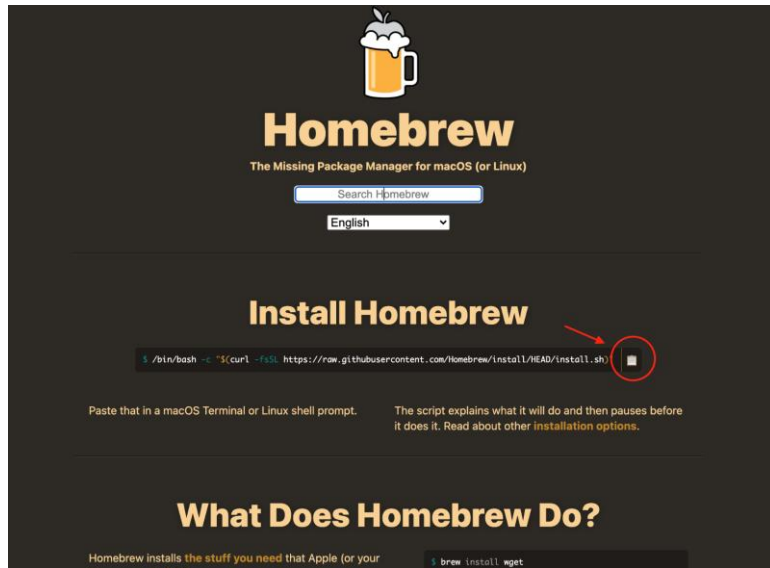
Portable ("thumbdrive edition")

[32-bit Git for Windows Portable.](#)

[64-bit Git for Windows Portable.](#)

<https://git-scm.com/>

Git 설치 (Mac)



<https://brew.sh/ko/> - Homebrew 설치 후 터미널에서 **brew install git**

Git 연결을 위한 초기 설정

Git 첫 설치 후 딱 한 번만 실행

TERMINAL

```
git config --global init.defaultBranch main  
git config --global user.name "프로필 이름"  
git config --global user.email "이메일 주소"  
git config --global --list
```

git init - 저장소 초기화

프로젝트 폴더에서 딱 한 번만 실행

Terminal

```
$ git init
```

현재 폴더를 Git 저장소로 만들었다는 뜻, git은 폴더별로 이력 관리를 한다.

실행하면 폴더 안에 숨김 폴더 .git/ 이 생기고, 여기에 모든 변경 이력이 저장됨

"empty"는 아직 커밋이 하나도 없는 빈 저장소라는 의미

.git 폴더의 의미

터미널에서 `ls -a` 명령어로 확인 가능한 숨겨진 폴더

.git = Git 저장소 그 자체

모든 커밋(스냅샷), 설정, 작업 히스토리가 이 폴더 안에 저장됨

⚠ 절대 수동으로 삭제·수정 금지

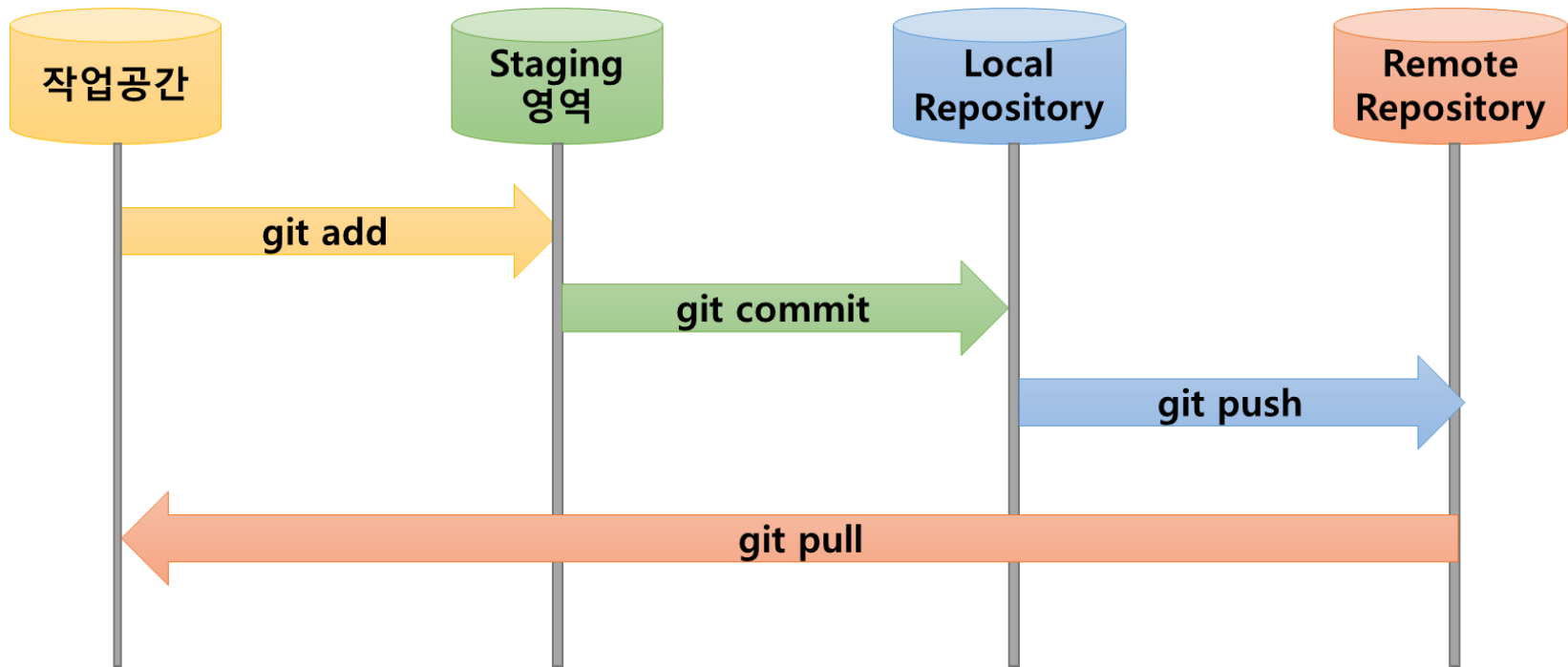
삭제 시 지금까지 저장한 모든 히스토리가 복구 불가능하게 사라짐

PART 3

Git 사용하기

posco

Git 작업 흐름



파일 상태와 Git 작업 흐름 - 4가지 상태

Untracked Git이 아직 모르는 새 파일

Modified 추적 중인 파일이 수정된 상태

Staged git add 실행 후 - 다음 커밋에 포함할 파일을 Git에 알린 상태

Committed git commit 실행 후 - 스냅샷이 히스토리에 저장된 완료 상태

파일 상태와 Git 작업 흐름 - 상태 전환



git add → git commit → 파일 수정 → git add (반복 사이클)

* why add → commit 두 단계인가?

10개 파일을 수정했더라도 **3개만** 이번 커밋에 포함할 수 있음

1. 포함할 3개만 `git add`해서 Staged 상태로 올리기
2. `git commit`으로 그 3개만 커밋
3. 나머지 7개는 다음 커밋에서 별도 처리

git add - 스테이징에 올리기

Untracked·Modified 파일을 Staged 상태로 전환

Terminal

```
git add hello.txt  
git add .
```

`git add hello.txt`

특정 파일(hello.txt)만 스테이징에 올림

`git add .`

현재 폴더의 모든 변경사항을 스테이징에 올림

git commit - 스냅샷 저장

Staged 파일들을 스냅샷으로 확정 - Git 히스토리에 체크포인트 생성

Terminal

```
git commit -m "변경 내용을 설명하는 메시지"
```

스테이징에 올라간 변경사항을 **하나의 커밋(스냅샷)**으로 저장

-m 옵션은 커밋 메시지를 한 줄로 바로 입력할 때 사용

좋은 커밋 메시지 작성법

Q1

나쁜 커밋 메시지는?

A. '수정', 'fix', '업데이트', '오' - 무엇을 했는지 알 수 없는 메시지

Q2

좋은 커밋 메시지의 기준은?

A. 무엇을 했는지 명확히, 한 달 뒤에도 맥락을 알 수 있게

Q3

입문 단계에서 쓰기 좋은 형식은?

A. "동사 + 대상" - 'notes.txt 추가', '로그인 버튼 오류 수정'

git status - 현재 상태 확인

Changes to be committed 초록색 - Staged 상태, 커밋하면 이 파일들이 포함됨

Changes not staged 빨간색 - 수정됐지만 아직 add 안 한 파일

Untracked files 빨간색 - Git이 아직 알지 못하는 새 파일

git log - 커밋 히스토리 확인

모든 커밋을 시간 역순으로 표시 - q 키로 종료

Terminal

```
git log
```

```
commit eb5cf0bb7cfaf3195323b55b6d97304ba3cc1953
Author: allie <allie@spreatics.com>
Date:   Wed Dec 17 16:45:43 2025 +0900

    커밋메세지

commit 18f83b95c278a00ee2fc9fc9b537f805cdf5d52
Author: allie <allie@spreatics.com>
Date:   Tue Dec 9 12:51:36 2025 +0900

    커밋메세지

commit a7a9b05bbdfcb4750a9b2030699e0e50f5d28e54
Author: allie <allie@spreatics.com>
Date:   Fri Nov 21 16:00:09 2025 +0900

    커밋메세지

commit 25555f3401c8a2f70d639d04c64ab89d94b0ab90
Author: allie <allie@spreatics.com>
Date:   Fri Nov 21 11:51:56 2025 +0900

    커밋메세지
```

.gitignore - 추적 제외 파일 설정

프로젝트 루트에 만드는 텍스트 파일 - Git이 이 목록의 파일을 무시

Code

```
.env  
.DS_Store  
node_modules/  
*.log
```

.gitignore - 추적 제외 파일 유형

민감한 설정 파일

.env에 담긴 API 키·비밀번호가 GitHub에 올라가면 전 세계 공개

시스템 자동 생성 파일

.DS_Store(맥), Thumbs.db(윈도우) - 프로젝트와 무관한 파일

라이브러리 폴더

node_modules - 수천 개의 파일이 저장소를 불필요하게 크게 만들

실습 - 첫 저장소 만들고 add·commit·status·log 체험

목표: git init부터 add→commit→log 흐름을 처음으로 직접 체험

케이스 A notes.txt: 오늘 배운 명령어 정리

케이스 B profile.txt: 이름·수강 이유·목표 작성

PART 4

GitHub

posco

GitHub란 무엇일까

Git과 GitHub의 차이

Git = 내 컴퓨터의 버전 관리 도구 / GitHub = Git 저장소를 올려두는 플랫폼

GitHub로 가능한 것

코드 온라인 백업 + 어디서든 내려받아 작업 + 팀원과 공유·협업

로컬 저장소 vs 원격 저장소

로컬 저장소

내 컴퓨터의 Git 저장소 - 인터넷 없이도 커밋·log 가능

내 컴퓨터의 프로젝트 폴더

원격 저장소

GitHub 서버에 있는 Git 저장소 - 로컬과 독립적으로 존재

GitHub의 Repository

동기화 방법

git push로 로컬→원격, git pull로 원격→로컬

로컬 저장소 vs 원격 저장소 - 핵심

로컬에서 커밋한다고 자동으로 GitHub에 올라가지 않음 - 반드시 `git push` 실행 필요

GitHub에서 직접 파일을 수정해도 내 컴퓨터에 자동 반영 안 됨 - 반드시 `git pull` 필요

두 저장소는 독립적 - `push`와 `pull`로 명시적으로 동기화

- `push` 명령어 - 로컬 컴퓨터에서 GitHub의 Repository로 변경사항을 보내는 명령어
- `pull` 명령어 - 로컬 컴퓨터의 폴더와 연결된 GitHub Repository에서 로컬 컴퓨터로 변경사항을 가져오는 명령어

GitHub에서 Repository 만들고 가져오기



Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere?

[Import a repository.](#)

Owner *

linda-spr ▾

Repository name *

/

Great repository names are short and memorable. Need inspiration? How about [potential-octo-robot?](#)

Description (optional)

☒ Public

Anyone on the internet can see this repository. You choose who can commit.

☐ Private

You choose who can see and commit to this repository.

Initialize this repository with:

Skip this step if you're importing an existing repository.

☐ Add a README file

This is where you can write a long description for your project. [Learn more.](#)

Add .gitignore

Choose which files not to track from a list of templates. [Learn more.](#)

.gitignore template: None ▾

Choose a license

A license tells others what they can and can't do with your code. [Learn more.](#)

1단계

Repository name 입력 (예: my-git-lab), Public 선택

2단계

⚠ Add a README file 체크박스 - 체크 안 함

3단계

Create repository 클릭 → 생성된 HTTPS URL 복사

Repository 가져오기

HTTPS

SSH

<https://github.com/linda-spr/abcd.git>



TERMINAL

```
echo "# abcd" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/linda-spr/abcd.git
git push -u origin main
```

git remote add origin - 원격 저장소 연결

로컬 저장소에 GitHub 주소 등록 - 프로젝트당 딱 한 번

Terminal

```
git remote add origin https://github.com/사용자이름/저장소이름.git  
git remote -v
```

git push - 원격에 업로드

처음에는 -u 옵션, 이후에는 git push만

Terminal

```
git push -u origin main
```

```
git push
```

git push - 확인

처음 push: `git push -u origin main` - 기본 원격을 설정하는 한 번의 과정

이후 push: `git push`만 입력 - `-u` 설정 덕분에 자동으로 `origin main`에 업로드

push 성공 후 - GitHub 저장소 페이지 새로고침하면 파일과 커밋 히스토리 확인 가능

Git 명령어

TERMINAL

```
git status
```

```
git add .
```

```
git status
```

```
git commit -m "study: first commit"
```

```
git log
```

```
git push origin main
```

실습 - GitHub 저장소 생성 + 원격 연결 + 첫 push

목표: my-git-lab 로컬 저장소를 GitHub에 연결하고 처음으로 push 완료

케이스 A/B 공통

실습 1~3에서 만든 my-git-lab 폴더 사용

실습 - Step 1: GitHub 저장소 생성

github.com 로그인 + 버튼 → New repository

저장소 이름 입력 my-git-lab, Public 선택

Add a README file 체크 안 함 (중요!)

Create repository 생성 후 나타나는 HTTPS URL 복사

실습 - Step 2-3: 원격 연결 + 첫 push

git remote add my-git-lab 폴더로 이동 → git remote add origin 복사한URL

git remote -v 연결 확인

git push -u origin main push 실행

PAT 인증 사용자 이름: GitHub 아이디 / 비밀번호: PAT 붙여넣기

실습 - Step 4 + 체크포인트

Step 4: GitHub에서 확인

브라우저에서 자신의 GitHub 저장소 페이지 열기

로컬에서 만든 파일들이 업로드됐는지 확인

Commits 클릭해서 커밋 히스토리도 확인

GitHub 저장소 페이지에 내 파일과 커밋이 보이면 완료

git clone - 원격을 로컬로 복사

GitHub 저장소 전체를 로컬로 복사 - origin 연결 자동 설정

Terminal

```
git clone https://github.com/사용자이름/저장소이름.git  
git clone URL 원하는폴더명
```

git pull - 원격 변경사항 가져오기

원격 최신 커밋을 로컬로 반영 - fetch + merge 자동 처리

Terminal

```
git pull origin main  
git pull
```

git pull - 사용 시점

새 작업을 시작하기 전 - 다른 컴퓨터나 팀원이 push한 내용이 있을 수 있음

작업 시작 전 git pull이 좋은 습관 - 나중에 협업할 때 충돌을 줄여줌

pull 후 git log --oneline으로 새 커밋이 로컬에 반영됐는지 확인

push와 pull의 흐름 비교 - 방향



push = 로컬 → 원격(밀어 올리기) / pull = 원격 → 로컬(당겨 내려받기)

push와 pull의 흐름 비교 - 상황별 명령어

상황	쓸 명령어
내 커밋을 GitHub에 올리고 싶다	git push
GitHub 최신 내용을 내 컴퓨터에 받고 싶다	git pull
GitHub 저장소를 처음 내 컴퓨터로 내려받고 싶다	git clone
로컬과 원격이 같은지 확인하고 싶다	git status + git log

README.md 기초 작성 - 마크다운 문법

GitHub가 자동으로 렌더링하는 마크다운 기본 문법

Terminal

```
# 큰 제목 (H1)
## 중간 제목 (H2)
- 목록 항목
**굵게** *기울여*
```

README.md 기초 작성 - 답을 내용

프로젝트 소개

이 저장소가 무엇인지 한두 줄로 설명

실행 방법

어떻게 설치하고 실행하는지 간단한 절차

작성자 정보

이름 또는 GitHub 아이디 - 포트폴리오로 활용 시 중요

PART 5

GitHub Desktop

posco

GitHub Desktop이란?



GitHub에서 공식 제공하는 **무료 데스크톱 Git 클라이언트**

터미널 명령어 대신 **클릭과 시각적 인터페이스**로 Git을 사용

내부적으로는 **동일한 .git 저장소**를 사용 → CLI와 언제나 자유롭게 병행

커밋 단위 **시각적 diff**, 부분 스테이징, 충돌 해결 UI 등 **초보자 친화적 기능** 제공

GitHub Desktop 다운로드 및 설치

공식 사이트에서 받아 **GitHub 계정으로 로그인**하면 바로 사용 가능

- desktop.github.com 접속 → 본인 OS에 맞는 **설치 파일 다운로드**

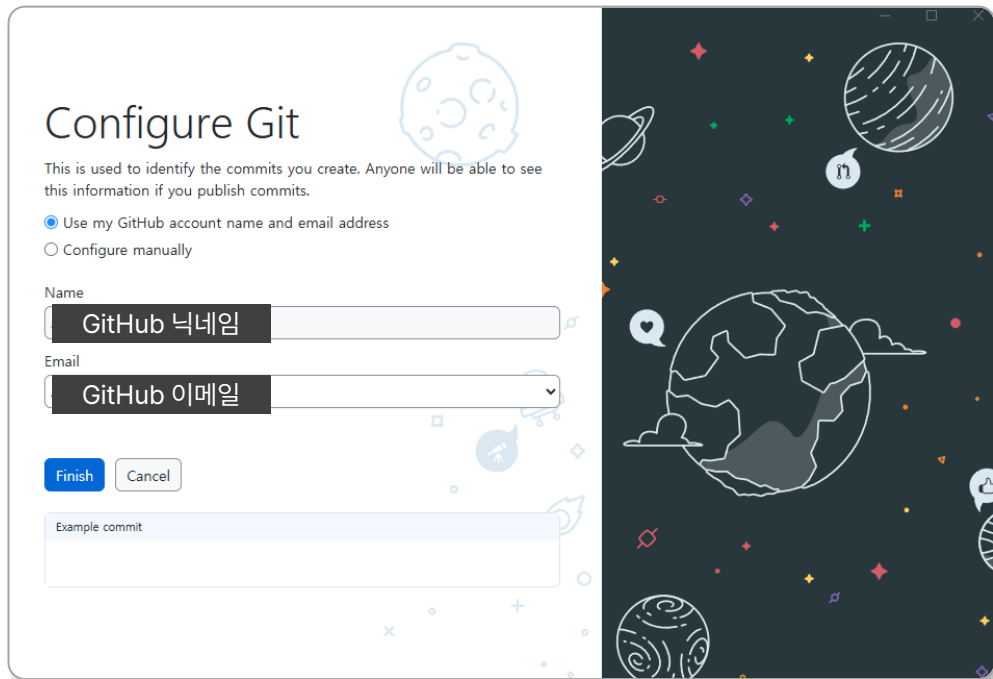
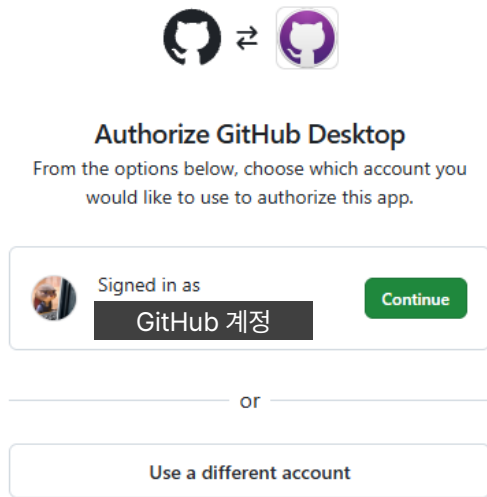
Download GitHub Desktop

Focus on what matters instead of fighting with Git. Whether you're new to Git or a seasoned user, GitHub Desktop simplifies your development workflow.

Download for Windows (64bit)

<https://desktop.github.com/download/>

GitHub Desktop 사용자 설정

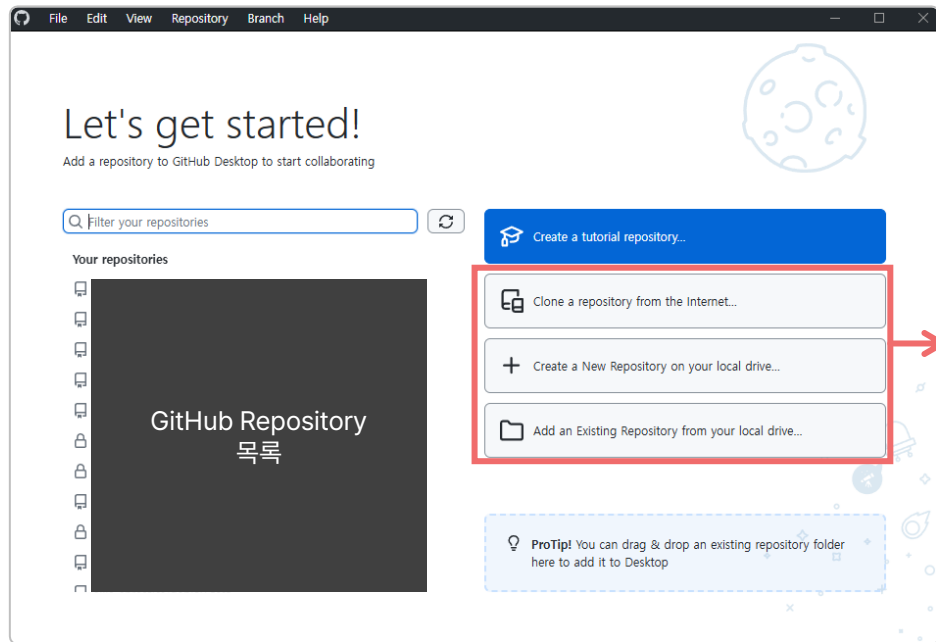


실행 후 "Sign in to GitHub.com" 선택
→ 브라우저 인증

최초 1회 사용자 이름·이메일 설정
:CLI의 git config와 동일한 정보

저장소 추가하기

상황에 맞는 세 가지 진입 경로로 저장소 시작



"Clone repository":

원격 저장소를 로컬로 복사 — CLI의 git clone과 동일

"Create new repository":

새 저장소를 만들어 GitHub에 Publish까지 한 번에 처리

"Add existing repository":

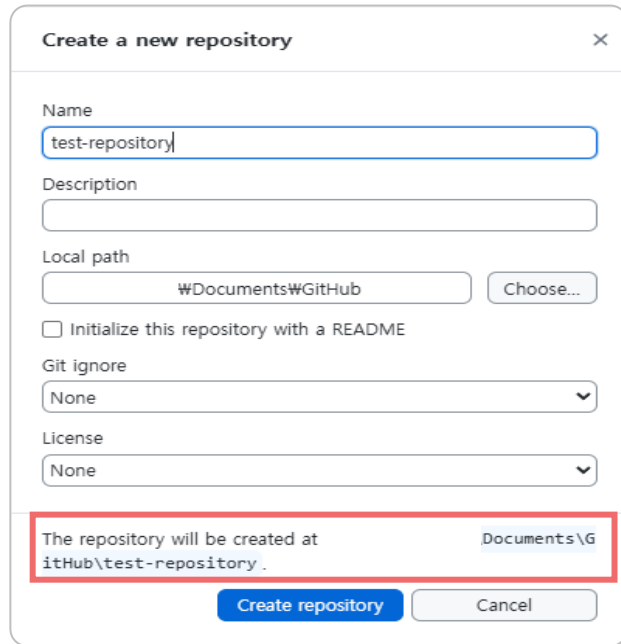
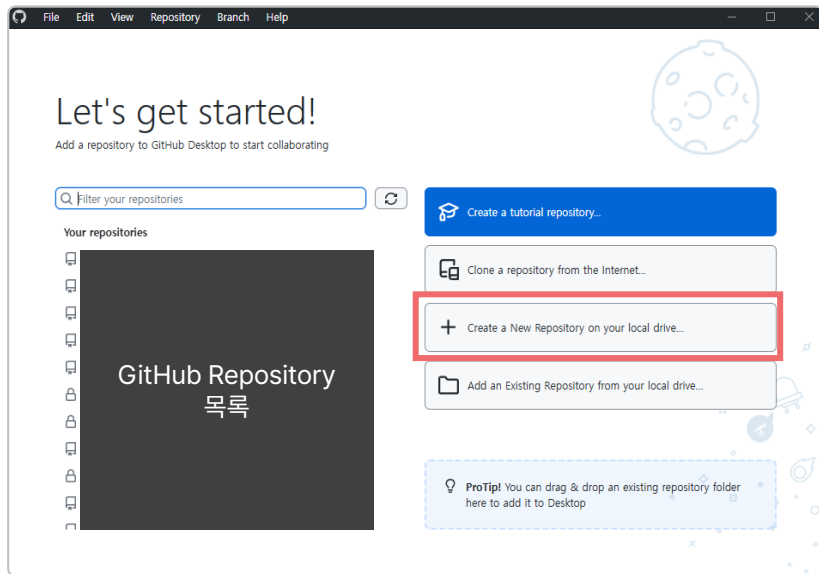
이미 있는 로컬 폴더를 GitHub Desktop에 연결

저장소 추가하기

로컬에 새로운 Repository 만들어보기

[Create a New Repository on your local drive]를 통해 만든 Repository는 로컬에서 만들어진다.

= .git/ 으로 관리되는 폴더가 하나 만들어진다.



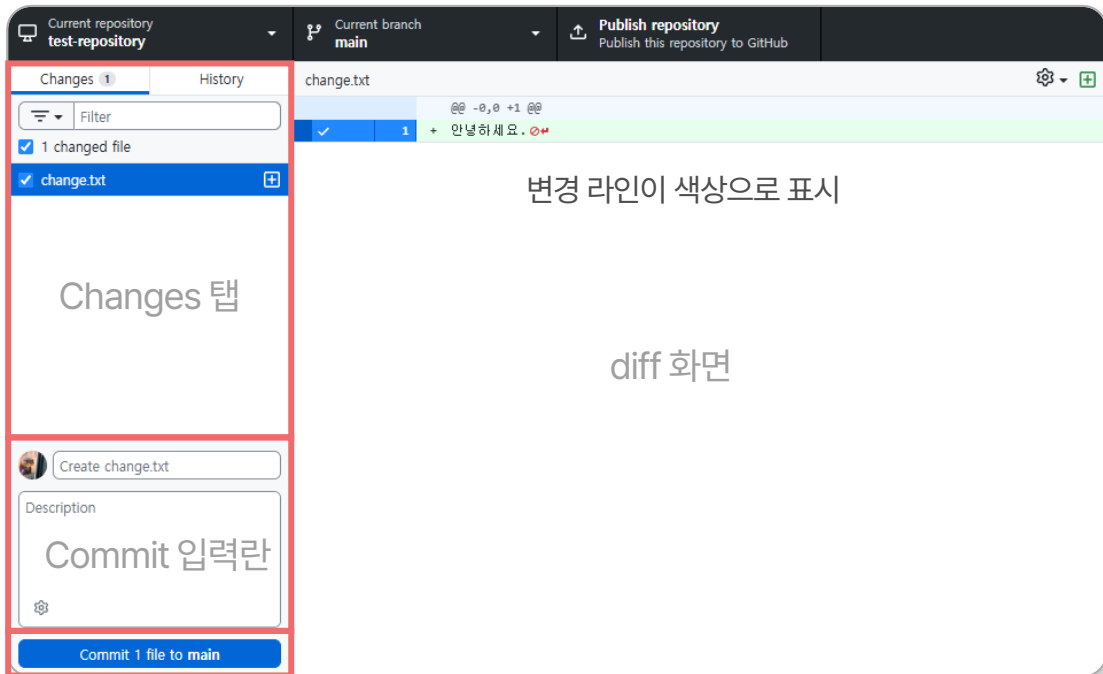
새로운 폴더(Local Repository) 가 만들어지는 경로 확인

변경사항 커밋하기

Changes 탭에서 **체크박스**로 스테이징하고 커밋 작성

- ① 좌측 **Changes** 탭에 수정된 파일이 자동 표시,
체크박스로 커밋 포함 파일 선택
→ **git add**와 동일

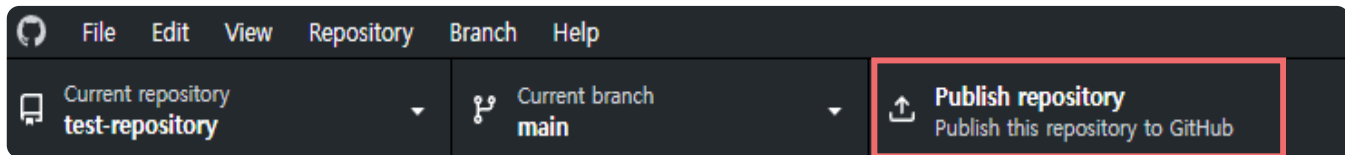
- ② 좌하단 **Summary**는 50자 이내,
핵심을 한 줄로 작성
→ **commit message** 작성



- ③ Commit 버튼 클릭 → **commit** 진행

원격 저장소 연결하기

로컬 저장소를 GitHub 원격과 연결하면 **Push/Pull**이 가능해짐



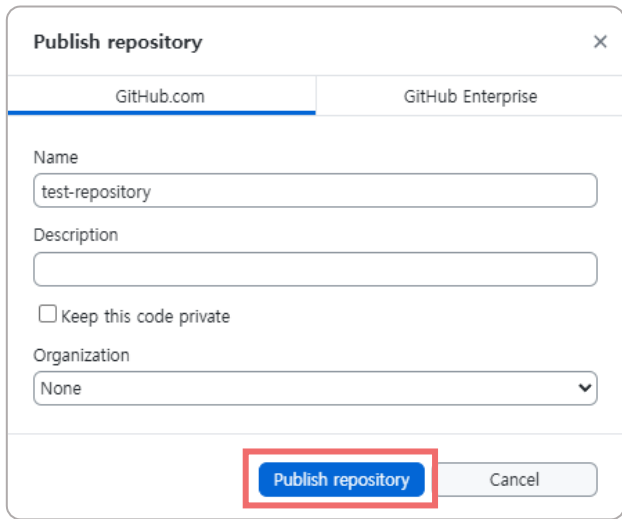
GitHub Desktop: 상단 "**Publish repository**" 버튼

→ GitHub 에 Repository를 만들고 push 작업을 동시에 진행할 수 있어요.

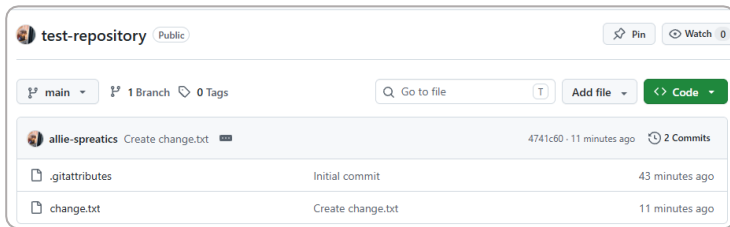
원격 저장소 연결하기

로컬 저장소를 GitHub 원격과 연결하면 **Push/Pull**이 가능해짐

"Publish repository" 버튼 클릭 후, → 저장소 이름·공개 여부 선택 후 **원격 생성 + 첫 Push**를 한 번에 처리

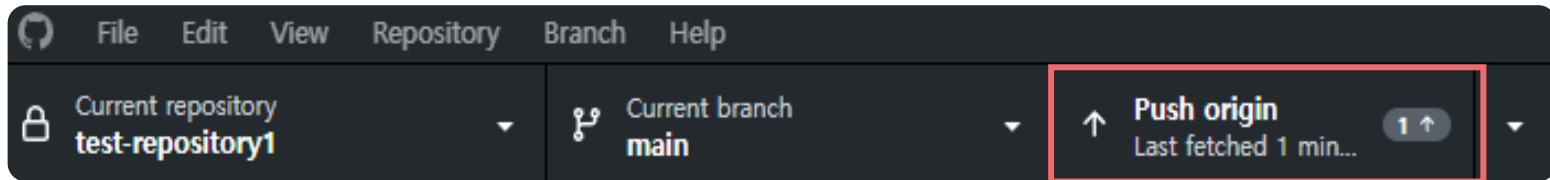


Publish repository 버튼을 누르면,
GitHub에 자동으로 Repository가 생성됨.



Push와 Pull로 원격 동기화

상단 단일 버튼이 상황에 따라 자동 전환



커밋 후 상단 버튼이 **"Push origin"**으로 바뀜 → 클릭 한 번으로 원격 업로드

원격에 새 커밋이 있으면 같은 자리에 **"Pull origin"** 표시 → 내려받기 안내

CLI vs GitHub Desktop

두 도구는 같은 .git을 공유 → 언제나 자유롭게 병행

공통

동일한 저장소를 Desktop에서 커밋 후 **터미널 git log**로 즉시 확인 가능

GUI 강점

시각적 diff, 라인 단위 부분 커밋, 협업자 온보딩에 유리

CLI 강점

rebase·cherry-pick·stash 등 고급 명령, 스크립트 자동화, 서버 작업

감사합니다.